

Secrets Management and Row Level Security

OpenBao vs HashiCorp Vault, a production-ready OpenBao design for our VMs, PostgreSQL Row Level Security, and Firebase Security Rules. A research report and reference guide.



Prepared by Hezki, technical research intern

For Production team

Date 8 October 2026

Versions checked OpenBao 2.7.1 · Vault Community 2.1.x · PostgreSQL 18 · PgBouncer 1.24 · Go + Fiber v3

Lab repository secrets-rls-lab

Contents

START HERE

S	Summary and recommendation	2
----------	----------------------------------	---

PART I · CHOOSING THE TOOL

1	The problem and the goal	4
2	OpenBao vs HashiCorp Vault	6

PART II · HOW OPENBAO WORKS

3	The big picture: components and startup order	9
4	Seal and unseal	12
5	The unsealer: transit auto-unseal	15
6	High availability with Raft	18
7	Networking and the load balancer	22
8	What OpenBao provides to an application	25

PART III · ROW LEVEL SECURITY

9	PostgreSQL Row Level Security	29
10	One request, end to end	33

PART IV · PUTTING IT INTO CODE

11	Integrating a Go service	35
-----------	--------------------------------	----

PART V · OPERATING IT

12	What breaks if... failure scenarios	38
13	Production deployment	40
14	Rollout plan, risks and open questions	43

PART VI · FIREBASE

15	Firebase: Security Rules	45
-----------	--------------------------------	----

APPENDICES

A	The lab: repository map and test results	48
B	Command cheat sheet	50

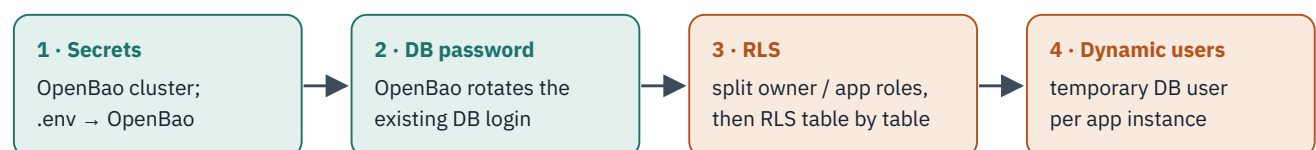
Summary and recommendation

We should adopt **OpenBao** as our secrets manager, run it as a 3-node cluster on our VMs with **transit auto-unseal**, and use **PostgreSQL Row Level Security** so the database itself keeps each customer's data apart. Firebase already has the equivalent for client apps: **Security Rules**.

The questions this research answers

Question	Answer
Vault or OpenBao?	OpenBao. Same API and tools as Vault, but truly open source (MPL 2.0) and community-governed. Features that Vault charges for are free: namespaces, reads from standby nodes, control groups. Vault's licence would allow our internal use, but it is not open source, and open source was the team's priority. <i>(Chapter 2)</i>
Can it run on our VMs (Tencent)?	Yes. We built and tested a production-like 3-node cluster behind a load balancer. It survived losing its leader and full restarts, and unsealed itself every time. <i>(Chapters 3–7)</i>
Is there a Tencent-specific problem?	One: Tencent sells a KMS, but neither OpenBao nor Vault has a seal for it, so it cannot unlock OpenBao automatically. The solution is transit auto-unseal , a small second OpenBao that holds the unlock key. Built in, and tested. <i>(Chapter 5)</i>
Can our <code>.env</code> files go away?	Yes. Apps log in to OpenBao at startup and read their secrets from it. Tested with an import script that moves an existing <code>.env</code> file into OpenBao. <i>(Chapter 8)</i>
Can the database password go away?	Yes. OpenBao can create a temporary database user for each app, which expires by itself. Works through PgBouncer. Tested: users rotated automatically with zero failed requests. <i>(Chapters 8–11)</i>
Does PostgreSQL RLS work with our setup?	Yes, including with one shared login and PgBouncer, because the app passes "who is asking" inside each transaction. One blocker today: our app login probably owns the tables, and table owners skip RLS. The roles must be split first. <i>(Chapters 9–10)</i>
And Firebase?	Security Rules do the same job for the web and Flutter apps. But our Go backend uses the Admin SDK, which bypasses all rules , so backend code must check permissions itself. <i>(Chapter 15)</i>
How much does it cost to run?	Small. The software is free; an idle node used about 34 MiB in the lab. Official sizing for a small production node is 2–4 cores and 8–16 GB RAM. The real cost is operations time: certificates, backups, upgrades, key holders. <i>(Chapters 2, 13)</i>

Recommended rollout, in short



Firebase rules review and tests can run in parallel at any time. Full plan with dependencies: chapter 14.

How to read this report

The chapters follow the order in which the system is built and used: first the choice of tool, then how OpenBao works from the inside out (sealing, unsealing, the cluster, the network), then what it gives an application, then Row Level Security, the code, operations, and finally Firebase. Each chapter can be read on its own.

Every chapter is written as a **reference for our team**. Our lab repository, `secrets-rls-lab`, is the worked example: everything described was built and tested there. Look for these boxes:

IN ONE PARAGRAPH

The chapter's main idea. Read only these for a quick overview.

IN OUR LAB

Where to find it in `secrets-rls-lab`, and what we observed when testing.

FOR PRODUCTION

What changes when this runs for real on our VMs.

WATCH OUT

Mistakes we made, or traps that are easy to fall into.

Each chapter ends with its **official sources** (OpenBao, HashiCorp, PostgreSQL, PgBouncer and Firebase documentation, checked October 2026). Statements marked *tested* were verified in the lab.

1 The problem and the goal

IN ONE PARAGRAPH

Today our secrets sit in `.env` files, every service shares one long-lived database password, and customers' data is kept apart only by `WHERE` clauses in Go. A secrets manager removes the first two risks. Row Level Security removes the third.

1.1 Where we are today

Today	Risk
Secrets (API tokens, signing keys, DB passwords) in <code>.env</code> files on each VM	The files get copied around, end up in backups and maybe in git history. Nobody can tell who read what. Changing a secret means editing files by hand on every server.
One shared PostgreSQL login for all queries	It is never rotated, because changing it would break everything at once. If it leaks, it opens all data. It probably also owns the tables, so Row Level Security could not protect anything.
Tenant filtering only in Go <code>WHERE company_id = ...</code>	One forgotten <code>WHERE</code> in one query leaks one customer's conversations to another.

1.2 Where we want to be

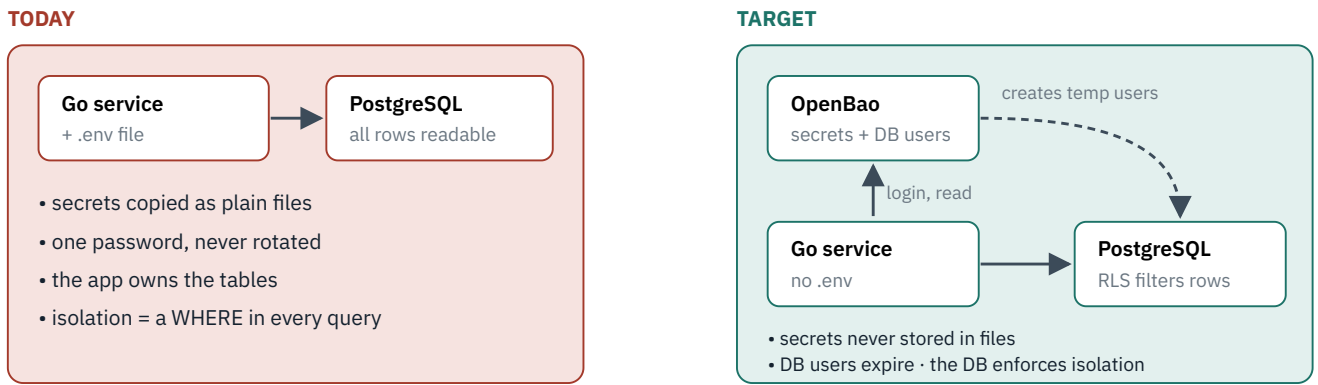


Figure 1. From secrets in files and isolation in code, to secrets in OpenBao and isolation in the database.

1.3 Words we use throughout

Term	Meaning
Secret	Anything that grants access: passwords, API tokens, signing keys.
Static / dynamic secret	Static: a stored value (an API token). Dynamic: created when requested and expires on its own (a temporary DB user).

Term	Meaning
Seal / unseal	OpenBao's data is encrypted. After every restart it is sealed (cannot read its own data) until its key is unlocked again.
Auth method	How a client proves who it is. AppRole for apps, OIDC or userpass for people.
Token	What a client gets after logging in. Carries policies and an expiry.
Policy	Rules on paths: what a token may read or write. Everything else is denied.
Secrets engine	A plugin mounted at a path: KV (stores values), database (creates DB users), transit (encrypts data)...
Lease	The lifetime of a dynamic secret. Renew it before it ends; at its maximum it must be replaced.
Raft	The built-in way OpenBao nodes keep identical copies of the data. One leader, the rest follow.
Row Level Security (RLS)	PostgreSQL rules attached to a table that decide which rows each query may see or change.

2 OpenBao vs HashiCorp Vault

IN ONE PARAGRAPH

OpenBao is a community fork of Vault 1.14, made just before Vault changed to a non-open-source licence. For what we need (stored secrets, app logins, policies, temporary DB users) the two are interchangeable: our scripts and Go code ran on both unchanged. OpenBao wins on licence, governance, and the features Vault only sells in its Enterprise edition. Vault wins on UI polish and paid vendor support. **Recommendation: OpenBao.**

2.1 Background

	OpenBao	HashiCorp Vault
Origin	Fork of Vault 1.14 (the last open-source line), taken just before the licence change	The original project
Licence	MPL 2.0: open source	BSL 1.1 (announced August 2023; first BSL release: Vault 1.15): source-available. Production use is allowed; the licence states that internal use within an organization is not a "competitive offering". Each version becomes MPL 2.0 four years after release
Owner	Community project under the OpenSSF (Linux Foundation)	IBM (acquisition of HashiCorp completed 27 Feb 2025)
Latest version	2.7.1 (1 Oct 2026); minor releases every few months, patch releases in between	2.1.2 Community (7 Oct 2026); our tests used 2.1.1. Vault moved from 1.21 to 2.0 in April 2026
Paid version	None. Third-party vendors sell support and managed OpenBao	Vault Enterprise (self-managed) and HCP Vault Dedicated (managed)
API	Same HTTP API (/v1/... , header <code>X-Vault-Token</code>), same CLI commands (<code>bao</code> vs <code>vault</code>), same Go client pattern	

2.2 Features: what is free where

✓ free · 💰 Vault Enterprise only · ✗ not available

Feature	OpenBao	Vault Community	Vault Enterprise
KV, database, transit, PKI, SSH, TOTP engines	✓	✓	✓
AppRole, JWT/OIDC, Kubernetes, userpass, certificate login	✓	✓	✓
Dynamic and static DB credentials (PostgreSQL, MySQL...)	✓	✓	✓
Raft HA cluster, manual snapshots	✓	✓	✓
Transit auto-unseal	✓ built in	✓	✓

Feature	OpenBao	Vault Community	Vault Enterprise
Login MFA (TOTP etc.)	✓	✓	✓
Namespaces (an isolated area per product)	✓ <i>tested</i>	\$ <i>tested: "enterprise-only feature"</i>	✓
Standby nodes serving reads	✓ (2.5+)	✗	\$
Control groups (a second person approves access)	✓ (2.7+)	✗	\$
PostgreSQL as storage backend	✓ production-ready; HA since 2.7	community-supported	community-supported
Cloud KMS / HSM auto-unseal	plugins (2.7+)	KMS ✓, HSM ✗	✓
Automated snapshots	✗ (use a scheduled job)	✗	\$
Replication between clusters (DR / performance)	✗	✗	\$
Sentinel, Transform, KMIP, Secrets Sync	✗	✗	\$
Web UI	basic	richer	richest

What we would miss from Vault Enterprise: replication between clusters, Sentinel policies, automated snapshots, and support from IBM. None of these is needed at our current scale.

2.3 Resources

	OpenBao 2.7.1	Vault 2.1.1	Source
Container image (compressed / on disk)	80 MB / 275 MB	188 MB / 740 MB	<i>tested</i>
Idle memory (dev mode)	34 MiB	37 MiB	<i>tested</i>
Small production node	2–4 cores, 8–16 GB RAM, 100+ GB disk, 3,000+ IOPS		HashiCorp reference architecture
Nodes	3 minimum for high availability, 5 recommended		Official docs

The resource needs are effectively the same. The cost that matters is operations time (see chapter 13).

2.4 Compatibility, tested

We ran the same setup scripts and the same Go API against both products. Only the environment variable names (`BA0_*` vs `VAULT_*`) and the CLI name changed. Policies gave identical results (allowed own path, denied another product, denied admin path). Only namespaces failed on Vault. **Switching later, in either direction, is low-risk.**

2.5 Things that changed in OpenBao 2.x

Older Vault tutorials are wrong on these points:

Change	What it means
<code>file</code> storage backend removed	Use Raft (integrated storage) even for a single node.
Audit devices are declared in the config file	<code>bao audit enable</code> through the API is refused; audit logging is versioned with the config.
<code>mlock</code> removed	No <code>IPC_LOCK</code> , no <code>disable_mlock</code> . Protect memory by disabling or encrypting swap on the VM.
HSM and cloud-KMS seals moved to plugins	Transit (our choice) is still built in.

DECISION

Choose **OpenBao**: open source (the team's priority), free namespaces for per-product isolation, smaller footprint, and the same API as Vault, so the decision is reversible.

OFFICIAL SOURCES

- [OpenBao home and FAQ \(fork origin\)](#) · [OpenBao licence \(MPL 2.0\)](#) · [Governance \(OpenSSF\)](#)
- [OpenBao 2.7 release notes](#) · [Deprecations](#) · [mlock removal RFC](#)
- [Vault licence \(BSL 1.1\)](#) · [HashiCorp BSL announcement](#) · [Vault Enterprise features](#)
- [Vault release notes](#) · [IBM completes HashiCorp acquisition](#)
- [HashiCorp Raft reference architecture \(sizing\)](#)

3 The big picture: components and startup order

IN ONE PARAGRAPH

The system has three layers. The **secrets layer** is a small unsealer plus three OpenBao nodes behind a load balancer. The **app** logs in to OpenBao at startup. The **data layer** is PostgreSQL behind PgBouncer, where OpenBao creates temporary users and RLS filters the rows. Things must start in a fixed order, and some state lives outside the servers (keys held by people), which is where most mistakes happen.

3.1 The three layers

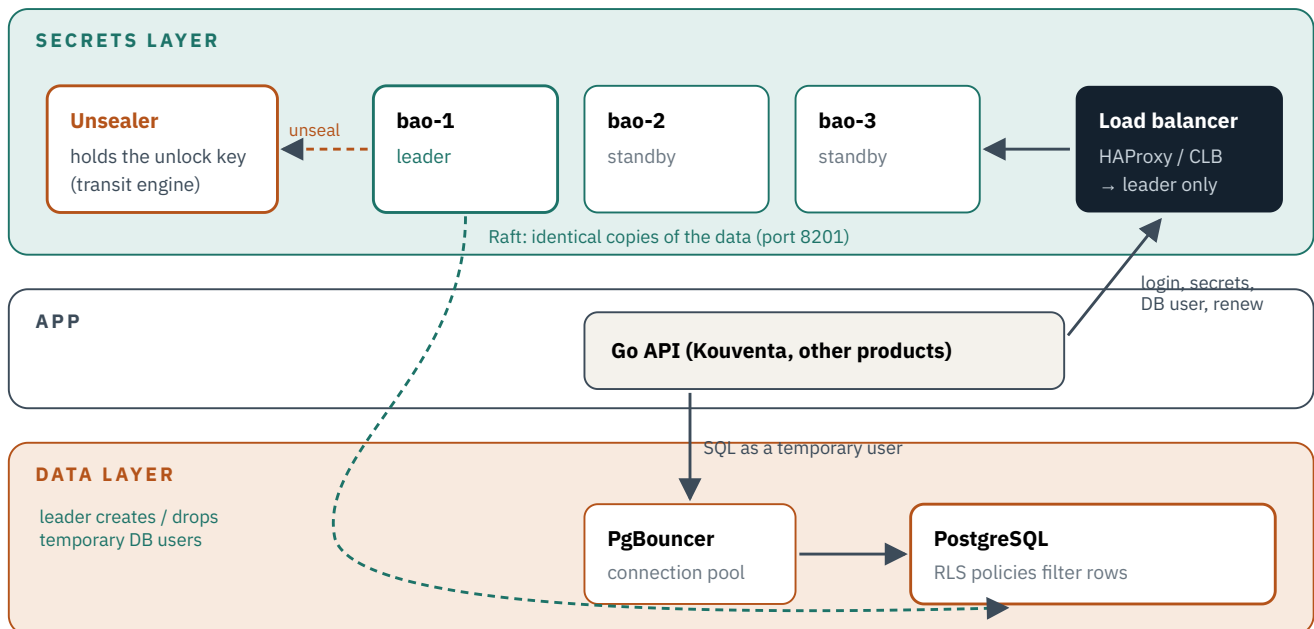


Figure 2. The three layers and who talks to whom. The app never talks to a specific OpenBao node, only to the load balancer.

Component	Job	Covered in
Unsealer	A small separate OpenBao. Holds the one key that unlocks the main cluster at startup.	Chapter 5
OpenBao nodes	Store secrets, check logins and policies, create DB users. One leader, the others standby.	Chapters 4, 6, 8
Load balancer	One address for all clients; always forwards to the current leader.	Chapter 7
Go API	Logs in at startup, reads secrets, gets a temporary DB user, keeps them alive.	Chapter 11
PgBouncer	Shares a few real database connections among many app connections.	Chapter 9

Component	Job	Covered in
PostgreSQL	The data, plus the RLS policies that decide which rows each request sees.	Chapters 9, 10

3.2 Startup order

Each piece depends on the one before it. If one step fails, everything after it waits.

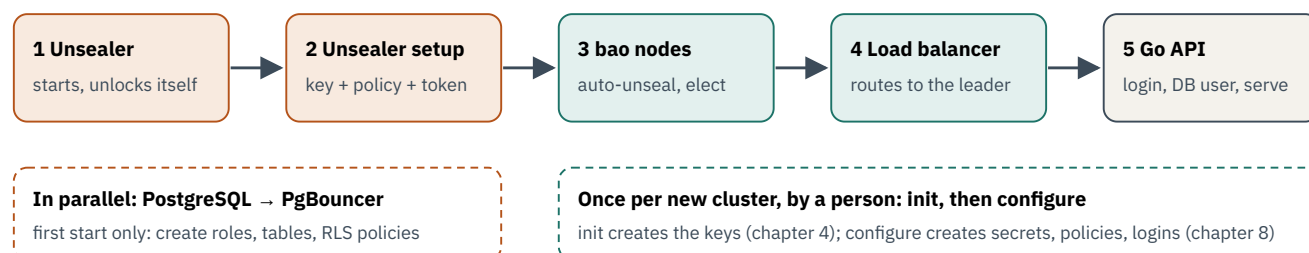


Figure 3. Startup order. Steps 1–5 happen on every start; the dashed boxes do not.

3.3 Where state lives, and why it matters

State	Lives in	If it is lost
Secrets, policies, logins	Each node's Raft data (encrypted)	Restore from a Raft snapshot (chapter 6)
The unlock key (transit key)	The unsealer's data only	Unrecoverable: the cluster can never unseal again. Back it up separately
Recovery keys, first root token	With people, created once at init	Rare admin tasks become impossible; split them among several people
App login (<code>role_id</code> , <code>secret_id</code>)	On the app server, delivered at deploy	Deliver a new <code>secret_id</code> (chapter 8)
Database rows and roles	PostgreSQL	Normal database backups

IN OUR LAB

The whole system runs as Docker containers from `reference/docker-compose.yml`. Compose enforces the startup order with `depends_on` rules: the nodes wait until `unsealer-setup` has *exited successfully*; PgBouncer waits until PostgreSQL is *healthy*.

Component	Container	Config	On your laptop
Unsealer	<code>unsealer</code> + one-shot <code>unsealer-setup</code>	<code>unsealer/unsealer.hcl</code> , <code>unsealer/setup.sh</code>	:8220
Nodes	<code>bao-1</code> , <code>bao-2</code> , <code>bao-3</code>	<code>openbao/bao-N.hcl</code>	:8201–8203
Load balancer	<code>haproxy</code>	<code>haproxy/haproxy.cfg</code>	:8300 (API), :8404 (stats)
Database	<code>postgres</code> , <code>pgbouncer</code>	<code>postgres/initdb/*.sql</code> , <code>pgbouncer/pgbouncer.ini</code>	:5433, :6432
Admin tools	<code>tools</code> (run on demand)	<code>scripts/init.sh</code> , <code>configure.sh</code> , <code>snapshot.sh</code>	—
App	Go API (not in Docker)	<code>api/</code>	:3000

WATCH OUT

`docker compose down -v` deletes the data volumes, but not files kept in normal folders. After a reset, the saved root tokens in `reference/.secrets/` belong to a cluster that no longer exists. Delete them and run `init` and `configure` again. We hit exactly this: the unsealer setup failed with a confusing *"Vault is sealed"* error.

OFFICIAL SOURCES

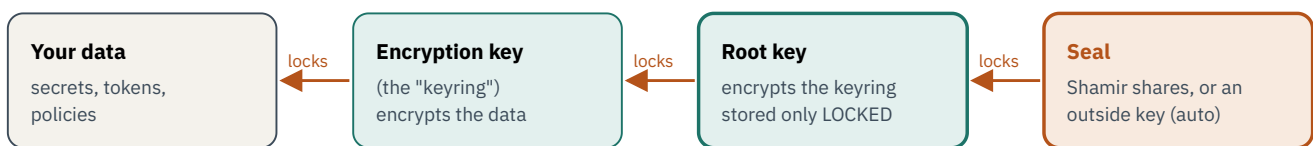
- [OpenBao: high availability](#) · [OpenBao: architecture](#)
- [Docker Compose: depends_on conditions](#)

4 Seal and unseal

IN ONE PARAGRAPH

Everything OpenBao stores on disk is encrypted, and the key that unlocks it is itself locked. A node that has just started is **sealed**: it can see its data but not read it. **Unsealing** unlocks the key chain into memory. It is done either by people typing key shares (Shamir) or automatically by a trusted outside service (auto-unseal). Because the unlocked keys only ever live in memory, **every restart seals the node again**.

4.1 The key chain



Unsealing runs the chain right to left: the seal unlocks the root key → the root key unlocks the keyring → the keyring decrypts data on demand. The unlocked keys stay in memory only, never on disk.

Figure 4. Three locks in a row. Only the seal is outside OpenBao's storage.

State	Memory holds	What the node can do
Sealed	nothing	Knows where its data is but cannot read it. Answers almost every request with <code>503 Vault is sealed</code> . Only status and unseal work.
Unsealed	root key + keyring	Decrypts data on demand and serves requests normally.

4.2 Initialization: once per cluster

`bao operator init` is run once, on one node of a brand-new cluster. It:

1. creates a random encryption key and a random root key;
2. locks the encryption key with the root key, and the root key with the seal;
3. stores both locked keys;
4. hands people the keys they will need later, plus the first **root token**.

DON'T CONFUSE

The **root key** is a cryptographic key inside the chain; no person ever sees it. The **root token** is an all-powerful admin *login* for API calls. Use it for the first setup only, then revoke it.

4.3 Ways to unseal

Seal	How it works	Init gives people...	Good for
Shamir (default)	The unlock key is split into shares (default 5); any threshold of them (default 3) rebuilds it. People type their share on every node, after every restart .	Unseal keys	Small setups; the root of trust is people
Static	A fixed 32-byte key from the config, an environment variable or a file. The node unseals itself.	Recovery keys	Only when another trusted source already protects that key
Transit (our choice)	The root key is locked by a key in <i>another</i> OpenBao's transit engine. The node asks it to decrypt at startup.	Recovery keys	Any VM provider; no cloud KMS needed
Cloud KMS / HSM	Same idea with AWS/GCP/Azure KMS or an HSM (plugins in 2.7+).	Recovery keys	Not available for Tencent's KMS

Recovery keys are not unseal keys. They *authorize* rare operations that need several people, such as generating a new root token. They can also authorize unsealing again after someone sealed a node by hand, but the actual decryption is still done by the seal. If the seal itself is lost, recovery keys cannot decrypt the root key.

4.4 What happens when a node starts (auto-unseal)

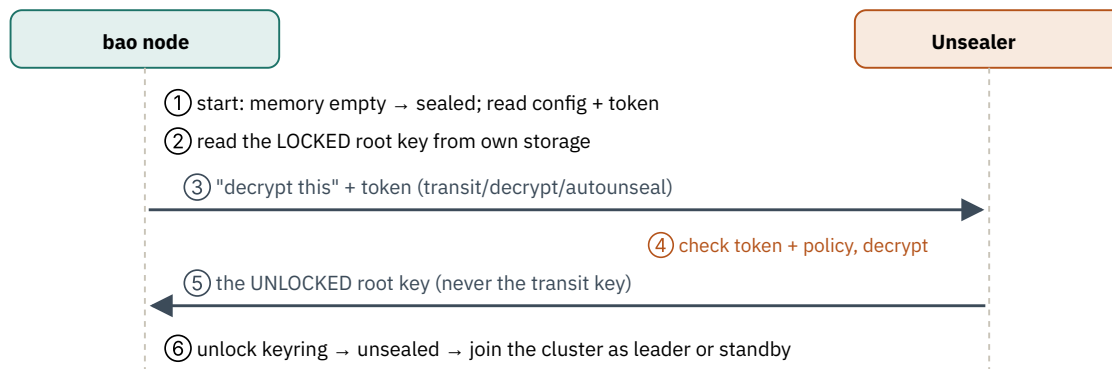


Figure 5. Auto-unseal with transit. This exchange happens once per node start; after that the node no longer needs the unsealer to serve requests.

IN OUR LAB

Each node's config (`reference/openbao/bao-N.hcl`) has the seal block below. The token is not in the file; OpenBao reads it from the `BAO_TOKEN` environment variable set in the compose file.

```
reference/openbao/bao-1.hcl

seal "transit" {
  address      = "http://unsealer:8200" # who to ask
  key_name     = "autounseal"           # which key to ask for
  mount_path   = "transit/"
}
```

Tested: `bao status` shows Seal Type `transit` and Recovery Seal `true`. Sealing the active node by hand (`bao operator seal`) made the load balancer switch to another node; restarting the sealed node made it unseal itself and rejoin as a standby.

FOR PRODUCTION

- Init with recovery keys split among **several people** (e.g. 5 shares, threshold 3), each encrypted with that person's PGP key (`-recovery-pgp-keys`). Never store them together or on the servers.
- Revoke the root token after the first setup. Create a new one with recovery keys only when needed.
- Plan for "every restart = sealed": monitoring should alert on any sealed node.

OFFICIAL SOURCES

- [OpenBao: seal/unseal concepts](#) (sealed state, key chain, Shamir, auto-unseal, recovery keys)
- [OpenBao: seal configuration](#) · [transit seal](#) · [static seal](#)
- [OpenBao: bao operator init](#)

5 The unsealer: transit auto-unseal

IN ONE PARAGRAPH

The unsealer is a second, very small OpenBao whose only job is to hold one encryption key (`autounseal`) in its **transit engine**. When a cluster node starts, it sends its locked root key to the unsealer, which decrypts it and sends back only the result. The key never leaves the unsealer. Each node proves who it is with a narrow token that may only encrypt and decrypt with that one key. We need this because Tencent's KMS cannot be used as an OpenBao seal.

5.1 Why we need it

Auto-unseal needs something *outside* the cluster to lock and unlock the root key. The usual options:

Option	For us
Cloud KMS (AWS, GCP, Azure...)	Not on our provider. Tencent sells a KMS, but OpenBao has no seal for it . (The <code>tccloudpublic</code> seal in the docs is T-Systems' T Cloud Public, not Tencent.)
HSM (hardware box)	Possible via a plugin, but expensive. Tencent Cloud HSM support would still need testing.
Transit: another OpenBao	Works anywhere, built in. Our choice.
Shamir (people type keys)	Works, but every node restart needs several people.

5.2 What the unsealer holds

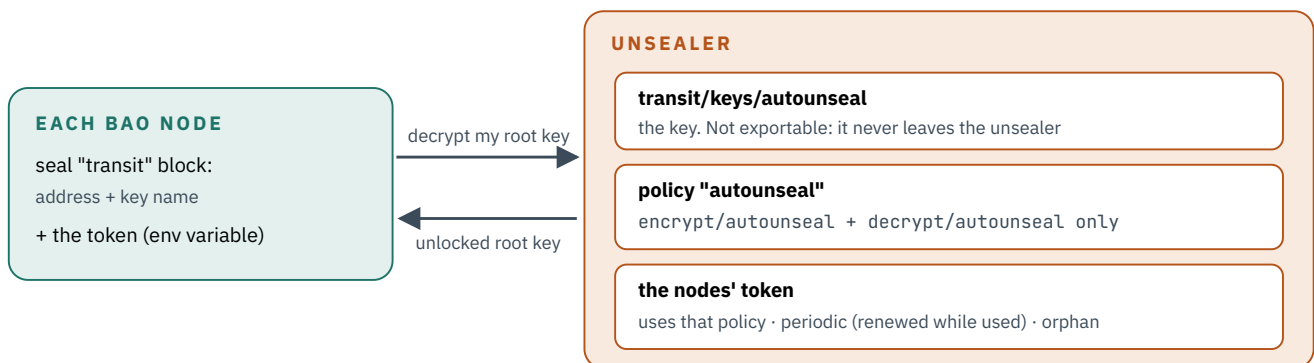


Figure 6. The unsealer stores exactly three things for the cluster: one key, one narrow policy, and the token that uses it.

5.3 Setting it up, step by step

The unsealer is an ordinary OpenBao server with its own storage. Setup is a short script:


```
reference/unsealer/setup.sh (simplified)

# A. first start only: initialize the unsealer itself, keep its admin token
bao operator init -recovery-shares=1 -recovery-threshold=1
# B. enable "encryption as a service"
bao secrets enable transit
# C. create the key (safe to repeat)
bao write -f transit/keys/autounseal
# D. a policy that allows ONLY encrypt and decrypt with that key
bao policy write autounseal - <<EOF
path "transit/encrypt/autounseal" { capabilities = ["update"] }
path "transit/decrypt/autounseal" { capabilities = ["update"] }
EOF
# E. the nodes' token, with that policy (only if it doesn't exist yet)
bao token create -policy=autounseal -orphan -period=24h
```

The token is the most exposed secret in the design: a copy sits on every node. That is why its policy is so narrow. A stolen token can ask "decrypt this", but it is useless without the node's locked root key and network access to the unsealer, and it cannot read, export or change anything.

5.4 Two kinds of calls: decrypt once, renew often

	Decrypt the root key	Renew the token
When	Once, every time a node starts	Periodically, while the node runs
Uses the transit key?	Yes	No
Purpose	Become unsealed	Keep the token valid for the <i>next</i> start

A **periodic token** has no end date, only a deadline that resets every time it is renewed. With a 24-hour period it lives forever as long as some node renews it at least once a day. If **no node runs for longer than the period** (for example, everything is shut down over a long weekend), the token expires and is deleted. The next start then fails with `403 permission denied` until a new token is created.

5.5 What breaks, and when

Situation	Nodes already running	A node that (re)starts
Unsealer down	keep serving (keys already in memory)	cannot unseal (connection error)
Nodes' token expired	—	403 permission denied; recreate the token
Unsealer data wiped, saved admin token still on disk	—	setup fails with "Vault is sealed"; full reset needed
Transit key lost for good	keep serving until they restart	never again: the cluster is unrecoverable

WATCH OUT

The unsealer is only needed at startup, so it is easy to forget. But losing its key means losing every secret in the main cluster. **Back it up separately from the cluster**, and test the restore.

IN OUR LAB

The unsealer container unseals itself with a **static key** from the compose file, so the lab starts with one command. A one-shot container (`unsealer-setup`) runs `setup.sh` on every `docker compose up` and exits; the nodes only start after it exits successfully. The token has a fixed, known ID (`lab-transit-unseal-token`), written in two places: where it is created (the setup script) and where it is used (the nodes' `BAO_TOKEN`).

Tested: stopping the unsealer did not affect the running cluster. A node restarted during that time could not unseal until the unsealer was back. After the laptop was off for more than 24 hours, the nodes failed with 403 until the token was recreated.

FOR PRODUCTION

- Run the unsealer on **its own small VM** (or a small 3-node cluster), separate from the main cluster.
- Unseal the unsealer with **Shamir keys held by people** (or an HSM). It rarely restarts, so this is the one place where people still unseal. A static key in a config file is only acceptable when another trusted system protects that key.
- Use a **random** token (no fixed ID), one per node if possible, stored in a root-only file on each node VM (e.g. a `systemd EnvironmentFile` with mode 600). Use a longer period (e.g. 7–30 days) and alert when its time left gets low.
- TLS between nodes and the unsealer; firewall so only the bao nodes can reach it.
- Revoke the unsealer's root token after setup.

OFFICIAL SOURCES

- [OpenBao: transit seal](#) · [transit secrets engine](#) · [static seal](#) (and its warning about when to use it)
- [OpenBao: tokens](#) (periodic and orphan tokens) · [policies](#)
- [OpenBao: tcloudpublic seal](#) (T-Systems, not Tencent) · [auto-unseal plugins RFC](#)

6 High availability with Raft

IN ONE PARAGRAPH

OpenBao keeps identical copies of its data on every node using **Raft**, a well-known agreement protocol. One node is the **leader** and handles writes; the others are followers (standbys). Every change is copied to the followers, and it only counts once a **majority** has saved it. If the leader dies, the others elect a new one within seconds. With 3 nodes the cluster survives losing 1; with 5, it survives losing 2. We don't implement Raft ourselves: it is built in, and we only configure it.

6.1 Roles and terms

Role	Does	In OpenBao terms
Leader	The only node that accepts writes. Sends regular heartbeats ("I'm alive") and log entries to the others.	active node
Follower	Copies what the leader sends. Never decides on its own.	standby node
Candidate	A follower that stopped hearing heartbeats and asks the others for votes.	during an election

Time is divided into numbered **terms**. Each term has at most one leader. A newer term always wins, so an old leader that comes back immediately steps down.

6.2 How a write is replicated

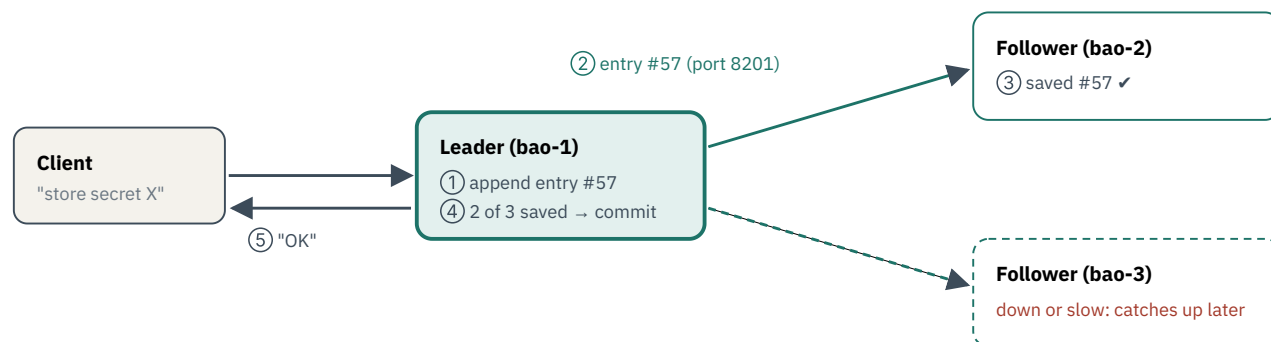


Figure 7. A write is committed when a majority (2 of 3) has saved it. The client gets "OK" only after that, so any single node can fail without losing it.

A node that was away receives the entries it missed when it returns, or a full **snapshot** of the data if it is far behind. This is also exactly how a brand-new node joins: it receives a snapshot, then follows the log.

6.3 Leader election

1. The leader sends heartbeats to the followers at short intervals.
2. If a follower hears nothing for a while (each node waits a slightly **random** time, so they rarely start together), it becomes a candidate, increases the term and asks for votes.

3. A node votes for it only if it hasn't voted in this term yet **and** the candidate's log is at least as up to date as its own. A node that missed recent writes therefore cannot become leader.
4. Votes from a majority make it the leader. It starts sending heartbeats.

6.4 Why majority is the key number

Any two majorities share at least one node. So whichever node wins an election, at least one of its voters already holds every committed write. It also means that at most one side of a network split can ever have a leader:



6.5 Configuring Raft in OpenBao

Raft is OpenBao's "integrated storage", enabled with a `storage "raft"` block. Each node gets the same file with its own name, data folder and the addresses of the others:

```
/etc/openbao/openbao.hcl on bao-1 (production form)

api_addr      = "https://bao-1.internal:8200" # my address for clients
cluster_addr = "https://bao-1.internal:8201" # my address for other nodes (Raft)

storage "raft" {
  path      = "/var/lib/openbao" # this node's own copy of the log + data
  node_id   = "bao-1"           # unique name in votes
  retry_join {
    leader_api_addr = "https://bao-2.internal:8200"
    leader_ca_cert_file = "/etc/openbao/tls/ca.pem"
  }
  retry_join {
    leader_api_addr = "https://bao-3.internal:8200"
    leader_ca_cert_file = "/etc/openbao/tls/ca.pem"
  }
}

listener "tcp" {
  address      = "0.0.0.0:8200" # API
  cluster_address = "0.0.0.0:8201" # Raft and node-to-node traffic
}
```

How the cluster forms

1. Start all nodes. They are uninitialized and keep retrying to join each other.
2. Initialize **one** node (chapter 4). It becomes a one-node cluster and its own leader.

3. On their next retry, the other nodes ask to join. The leader gives them a challenge they can only answer by unsealing through the **same seal**, which proves they belong. It then adds them as voters and sends them a snapshot.
4. From then on, every write is replicated as in Figure 7. Nobody runs init on the other nodes: they are initialized because they hold a copy of the data.

6.6 Operating the cluster

Task	How
See members and the leader	<code>bao operator raft list-peers</code>
Health of each member	<code>bao operator raft autopilot state</code>
A node dies temporarily	Nothing to do. A new leader is elected; the node catches up when it returns.
Replace a node permanently	<code>bao operator raft remove-peer <node_id></code> , then start a new VM with a new <code>node_id</code> ; it joins by itself. Autopilot can also clean up dead nodes automatically.
Upgrade without downtime	Upgrade and restart standbys one at a time; then <code>bao operator step-down</code> on the leader; then upgrade the old leader.
Back up everything	<code>bao operator raft snapshot save file.snap</code> . Restore with <code>... snapshot restore</code> . A snapshot is still encrypted: restoring it also needs the same seal (the unsealer).

IN OUR LAB

Three containers use `reference/openbao/bao-1.hcl` , `bao-2.hcl` and `bao-3.hcl` ; `scripts/init.sh` initializes `bao-1` only, and `scripts/snapshot.sh` saves and restores snapshots.

Tested: one write increased the "Raft Committed Index" equally on all three nodes. Stopping the leader produced a new leader within seconds while a write still succeeded with two nodes. The stopped node caught up after it restarted. Stopping two nodes stopped all service (no majority) until one came back. A full restart elected a leader in about 15 seconds.

FOR PRODUCTION

- **5 nodes** once we depend on it (3 to start), spread across availability zones, in the same region (every write waits for a majority).
- Local SSD for the Raft data; never a shared or network disk.
- Scheduled snapshots copied off the VMs (e.g. object storage). OpenBao has no automated snapshots, so use a timer job.
- Unique `node_id` per VM; never reuse the name of a removed node.

OFFICIAL SOURCES

- [OpenBao: integrated storage \(Raft\) internals](#) · [Raft storage configuration](#) (retry_join, autopilot settings) · [high availability](#)
- [OpenBao API: /sys/storage/raft](#) · [upgrading an HA cluster](#)
- [The Raft consensus algorithm](#) (the original paper and visual explanation)
- [HashiCorp Raft reference architecture](#) (node count, zones, sizing)

7 Networking and the load balancer

IN ONE PARAGRAPH

Clients only know one address: the **load balancer**. Every couple of seconds it asks each OpenBao node for its health, and only the active (leader) node answers `200`, so traffic always lands on the leader, even right after a failover. Nodes talk to each other on a separate cluster port (8201). Apps reach PostgreSQL through PgBouncer, while OpenBao reaches PostgreSQL directly to create users.

7.1 Every connection in the system

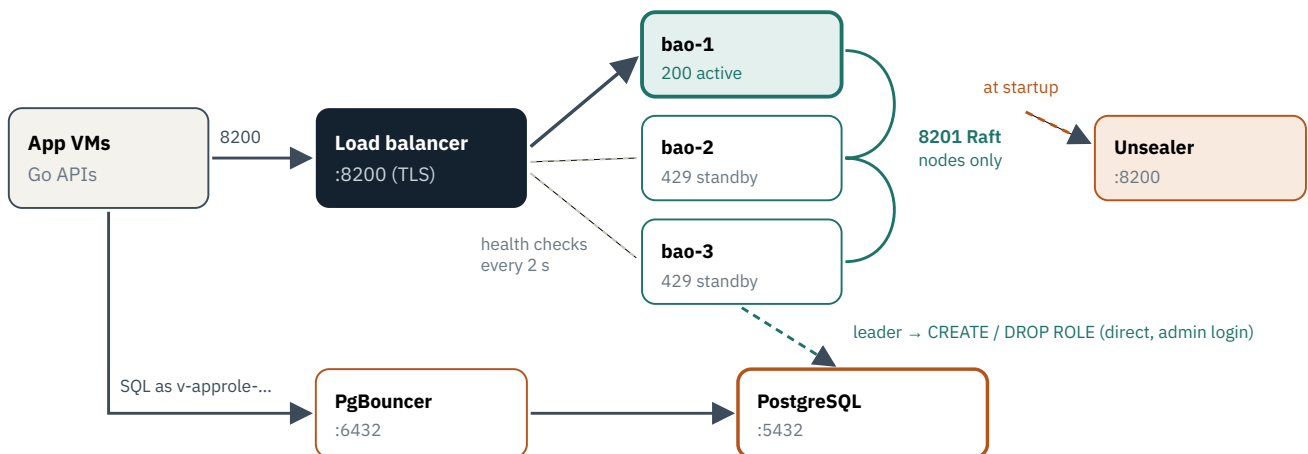


Figure 8. All connections. Solid arrows carry every request; dashed arrows are occasional (startup, user creation).

Port	On	Used by
8200 API (and UI)	each bao node, unsealer, load balancer	apps, admins, health checks, join requests, unseal requests
8201 cluster	each bao node	other bao nodes only: Raft votes, heartbeats, log entries, request forwarding
6432	PgBouncer	apps (as temporary users)
5432	PostgreSQL	PgBouncer, and OpenBao's database engine (admin work)

Port 8200 for the API and 8201 for cluster traffic is OpenBao's convention. All nodes use the same port numbers because each node is its own machine with its own address (`bao-1.internal:8201`, `bao-2.internal:8201` ...). Two settings control the cluster port: `cluster_address` in the listener (where to *listen*) and `cluster_addr` (what address to *announce* to the other nodes).

7.2 How the load balancer finds the leader

OpenBao's health endpoint `GET /v1/sys/health` returns a different HTTP status code per state:

Code	State	Load balancer
200	initialized, unsealed, active	send traffic here
429	unsealed standby	skip (healthy, just not the leader)
503	sealed	skip
501	not initialized	skip
no answer	process or VM down	skip

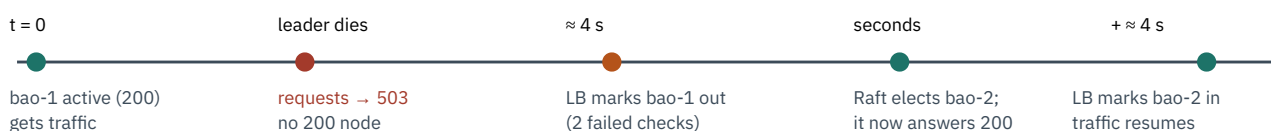
The load balancer is configured to accept only **200**. It knows nothing about Raft; the cluster simply tells it the truth. A healthy 3-node cluster therefore shows **one server up and two "down"**: for a load balancer, "down" means "don't send traffic here", not "broken".

```
reference/haproxy/haproxy.cfg

frontend openbao
  bind :8300
  default_backend openbao_active

backend openbao_active
  option httpchk GET /v1/sys/health      # ask each node
  http-check expect status 200           # only the active node passes
  default-server check inter 2s fall 2 rise 2 # every 2 s; out after 2 fails, in after 2 passes
  server bao-1 bao-1:8200
  server bao-2 bao-2:8200
  server bao-3 bao-3:8200
```

7.3 A failover, second by second



A gentle hand-over (bao operator step-down, e.g. for upgrades) is shorter than a crash, because the leader steps aside on purpose.

Figure 9. The short gap of 503 responses during failover. Apps should retry; ours do so on their next renewal cycle.

IN OUR LAB

Everything runs on one laptop, so there are two "worlds". **Inside** Docker's network, containers reach each other by service name and real port (`haproxy:8300` , `bao-1:8200` , `postgres:5432`). From the **laptop**, only published ports work, on `127.0.0.1` . The Go API runs on the laptop, so it uses `127.0.0.1:8300` (HAProxy) and `127.0.0.1:6432` (PgBouncer); the admin scripts run inside Docker, so they use `haproxy:8300` . The laptop ports 8201–8203 are just debug doors to each node's API (8200), and are not the cluster port. The HAProxy stats page (:8404) shows the reason for each "down" in its `LastChk` column (e.g. `L7STS/429` = standby).

Tested: while the active node was stopped, 75 of 75 API requests succeeded and `/status` showed the new active node.

FOR PRODUCTION

- A Tencent CLB or an HAProxy VM with the same health rule: `/v1/sys/health` , `success = HTTP 200` only (on CLB: status class `http_2xx` only).
- TLS on every listener. Security groups: 8200 from the load balancer, app subnets and the admin bastion; 8201 only between bao nodes; the unsealer's 8200 only from bao nodes.
- The "two worlds" disappear: every VM has a real private address.

OFFICIAL SOURCES

- [OpenBao API: /sys/health](#) (status codes) · [TCP listener](#) (address, cluster_address, TLS)
- [OpenBao: high availability](#) (active/standby, request forwarding)
- [Tencent CLB health checks](#)

8 What OpenBao provides to an application

IN ONE PARAGRAPH

For each service we configure four things, as code: a **KV store** with its static secrets (what used to be in `.env`), a **policy** saying exactly which paths it may use, an **AppRole** login so it can authenticate, and a **database role** from which OpenBao creates temporary PostgreSQL users on demand. Everything not explicitly allowed is denied.

8.1 The pieces for one service

```
what the configure script creates for "kouventa"

secret/                                KV v2 engine: static secrets
├─ kouventa/app                        jwt_signing_key, meta_api_token
├─ kouventa/firebase                  service_account_json
├─ kouventa/admin/break-glass         people only, never the app
└─ kouventa/db-owner                 LAB ONLY: owner login for the owner-trap demo

policy kouventa-app                    what the service's token may do
auth/approle/role/kouventa-api        how the service logs in → token with that policy

database/                             database engine: dynamic users
├─ config/supportdesk                how OpenBao connects to PostgreSQL (admin login)
└─ roles/kouventa-app                template for temporary users
```

8.2 Reading any OpenBao path

Everything in OpenBao is a **path**, and every path is also an HTTP address: `https://<server>:8200/v1/<path>`. A path has three parts:



Engine	Its routes (from its API docs)
KV v2 (secret/)	data/ values · metadata/ names and versions
Database (database/)	config/ connection · roles/ templates · creds/ read = create a user · rotate-root/ · static-roles/ , static-creds/
Transit (transit/)	keys/ · encrypt/ · decrypt/
AppRole (auth/approle/)	role/ · login
System (sys/ , always present)	health , seal , policies/ , leases/renew , leases/revoke ...

WATCH OUT

The KV command hides a word. `bao kv get secret/kouventa/app` really calls `GET /v1/secret/data/kouventa/app`. Policies must use the real path, which is why they say `secret/data/...`. To see the real call behind any CLI command, add `-output-curl-string`.

8.3 Policies: deny by default

```
policy kouventa-app
path "secret/data/kouventa/*" { capabilities = ["read"] } # ① own secrets
path "secret/metadata/kouventa/*" { capabilities = ["list"] } # ② list their names
path "secret/data/kouventa/admin/*" { capabilities = ["deny"] } # ③ except admin/
path "database/creds/kouventa-app" { capabilities = ["read"] } # ④ get a temporary DB user
path "sys/leases/renew" { capabilities = ["update"] } # ⑤ keep DB users alive
```

Rule	Effect
Deny by default	Anything not listed is forbidden: another product's secrets → 403.
The most specific path wins	③ is more specific than ①, so <code>kouventa/admin/*</code> → 403.
<code>deny</code> overrides everything	No other policy can re-allow a denied path.
Capabilities = HTTP methods	<code>read</code> GET · <code>create</code> / <code>update</code> POST/PUT · <code>delete</code> DELETE · <code>list</code> LIST
Every token also has <code>default</code>	The built-in default policy lets a token look up and renew itself .

8.4 AppRole: how a service logs in

AppRole is a login for machines: a **role_id** (like a username, not secret) and a **secret_id** (like a password). The login returns a token carrying the role's policies.

```
role configuration
bao write auth/approle/role/kouventa-api \
  token_policies="kouventa-app" \
  token_period=1h token_ttl=0 token_max_ttl=0 \ # periodic token (lab: 2m)
  secret_id_ttl=24h secret_id_num_uses=0
```

The token is **periodic**: the service renews it regularly and it never reaches an end date. This matters for the next section.

The `secret_id` is "secret zero": the one secret that cannot be removed, only limited. The service needs it only to log in at startup.

8.5 Dynamic database users

OpenBao logs in to PostgreSQL with a dedicated admin role and creates users from a template:

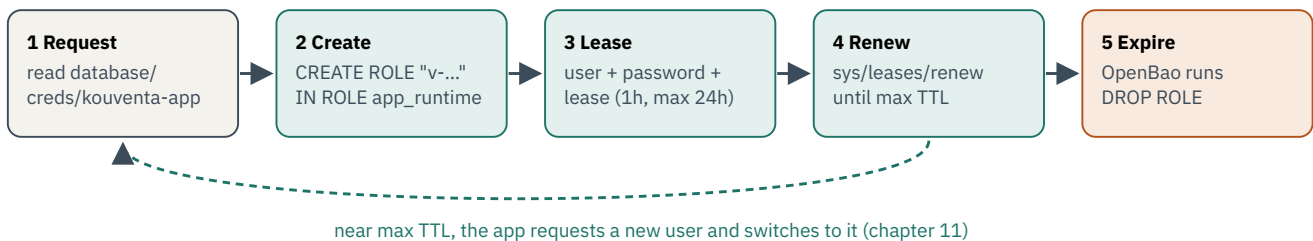
```

database engine setup

bao write database/config/supportdesk \
  plugin_name=postgresql-database-plugin allowed_roles=kouventa-app \
  connection_url="postgresql://{{username}}:{{password}}@postgres:5432/supportdesk" \
  username=openbao_admin password="initial-only"
bao write -f database/rotate-root/supportdesk      # new random password: only OpenBao knows it

bao write database/roles/kouventa-app db_name=supportdesk \
  creation_statements="CREATE ROLE \"{{name}}\" WITH LOGIN PASSWORD '{{password}}' \
    VALID UNTIL '{{expiration}}' IN ROLE app_runtime;" \
  default_ttl=1h max_ttl=24h                        # lab: 2m / 6m

```



Also expires early if the token that requested it expires or is revoked.

Figure 10. The life of a temporary database user.

- Each user is a **member of** `app_runtime`, so it gets exactly the app's rights, and RLS applies to it (chapter 9).
- `openbao_admin` is **not a superuser**: it can create and drop roles and add them to `app_runtime`, and RLS still applies to it. Note that on PostgreSQL 16+ the `ADMIN OPTION` grant also makes it a member of `app_runtime` by default, so it could use the app's rights. Grant it `WITH ADMIN OPTION, INHERIT FALSE, SET FALSE` so it can manage members but never read the data (*tested*).
- The username shows who asked: `v-approle-kouventa-...` for the service, `v-root-...` for an admin with the root token. Good for audits.

WATCH OUT: FOUND BY TESTING

A temporary DB user is tied to the **token that requested it**. When that token expires, OpenBao revokes the DB user too. With a normal (non-periodic) token, every re-login cut off the service's database user. The fix is the **periodic token** in 8.4.

IN OUR LAB

All of this is created by `reference/scripts/configure.sh` (safe to re-run), and the policy lives in `reference/openbao/policies/kouventa-app.hcl`. `reference/scripts/import-env.sh` moves an existing `.env` file into KV, one key per line.

Tested with `reference/scripts/app-check.sh`, logged in as the service: own secret **readable**; admin path **403**; another product **403**; writing its own secret **403**; temporary DB user **created** (`v-approle-kouventa-...`, lease 2m). Because that test token was never renewed, the DB user disappeared from PostgreSQL a few minutes later. The initial `openbao_admin` password no longer worked after `rotate-root`.

FOR PRODUCTION

- One policy and one AppRole per service. Consider one **namespace** per product for stronger isolation (free in OpenBao).
- `secret_id` delivered by the deploy pipeline, **response-wrapped**, single-use, limited to the app servers' IP ranges.
- Longer TTLs (e.g. token period 1h, DB users 1h / max 24h). Start with a **static role** (OpenBao rotates the existing login's password) before moving to dynamic users.
- People log in with OIDC or userpass and an admin policy, never with the root token.

OFFICIAL SOURCES

- [OpenBao: KV v2](#) · [policies](#) · [AppRole](#) · [tokens](#) · [leases](#)
- [OpenBao: database secrets engine](#) · [database engine API](#) · [PostgreSQL plugin API](#)
- [OpenBao API: /sys/leases](#)

9 PostgreSQL Row Level Security

IN ONE PARAGRAPH

Row Level Security (RLS) attaches rules called **policies** to a table. PostgreSQL adds them to every query on that table, so a forgotten `WHERE company_id = ...` can no longer leak data. The app tells PostgreSQL who is asking at the start of each transaction, and the policies use that. Three conditions make it work: the app must **not** log in as the table owner, the "who" must be set **per transaction** (to stay safe with PgBouncer), and missing context must mean **no rows**, not all rows.

9.1 Roles: who owns, who runs

Role	Purpose	RLS applies?
<code>postgres</code>	superuser: setup and seeding only	no (superusers bypass)
<code>app_owner</code>	owns the schema and tables; runs migrations	no, unless <code>FORCE</code> is set
<code>app_runtime</code>	the app's rights; owns nothing	yes
<code>v-approle-...</code>	temporary users, members of <code>app_runtime</code>	yes
<code>openbao_admin</code>	creates/drops the temporary users	yes; grant it <code>INHERIT FALSE, SET FALSE</code> so it cannot read data at all (8.5)
<code>pgbouncer_auth</code>	only for PgBouncer's password lookups	never reads data

MOST IMPORTANT RULE

The role that **owns** the tables must not be the role the app runs as. Table owners, superusers and roles with `BYPASSRLS` skip RLS completely. Today our app login probably owns the tables, so this split has to come first.

Grants and RLS work together: **grants** decide which *actions* a role may take (e.g. `SELECT`, `INSERT`, `UPDATE` but no `DELETE`); **RLS** decides which *rows* those actions touch. Every protected table carries `company_id` on itself (no joins needed in policies) and has an index on it.

9.2 Telling PostgreSQL who is asking

At the start of each transaction, the app sets three values. Small helper functions read them back for the policies:

```

per transaction, sent by the app
BEGIN;
SELECT set_config('app.company_id', '<uuid>', true),
       set_config('app.user_id',      '<uuid>', true),
       set_config('app.role',         'agent', true); -- true = this transaction only
-- ... the app's normal queries, with no WHERE company_id ...
COMMIT; -- the three values disappear

```

helper function (one per value)

```

CREATE FUNCTION support.current_company_id() RETURNS uuid LANGUAGE sql STABLE AS
$$ SELECT NULLIF(current_setting('app.company_id', true), '')::uuid $$;

```

- `current_setting(..., true)` returns NULL instead of an error when nothing was set.
- On a reused (pooled) connection, an old value comes back as an empty string. `NULLIF(..., '')` turns that into NULL too.
- NULL never equals anything, so **no context means no rows**. A bug shows up as an empty page, not as a leak.

9.3 Policies: walls and doors

Turn RLS on with `ALTER TABLE ... ENABLE ROW LEVEL SECURITY`. From then on, for every role except owners and superusers, a row is invisible unless a policy allows it. Policies come in two kinds:

Kind	Combined with others by	Think of it as
RESTRICTIVE	AND: every one must pass	a wall (e.g. tenant isolation)
PERMISSIVE (default)	OR: at least one must pass	a door (e.g. "agents see their own")

row visible = all walls pass AND at least one door opens

```

reference/postgres/initdb/03-rls.sql (shortened)
ALTER TABLE support.conversations ENABLE ROW LEVEL SECURITY;

-- wall: companies never see each other
CREATE POLICY tenant_isolation ON support.conversations AS RESTRICTIVE FOR ALL
  USING (company_id = support.current_company_id())
  WITH CHECK (company_id = support.current_company_id());

-- doors for reading
CREATE POLICY agent_read ON support.conversations FOR SELECT
  USING (assigned_to = support.current_user_id() OR assigned_to IS NULL);
CREATE POLICY supervisor_read ON support.conversations FOR SELECT
  USING (support.current_app_role() = 'supervisor');

-- plus: agent_update (own rows; supervisors any) and create_in_own_company (inserts)

```



Ana (Acme, agent). Sari (Acme, supervisor) passes the same wall, but the supervisor door opens for her: she gets all 5 Acme rows.

Figure 11. The wall keeps companies apart; the doors decide what each role sees inside the company.

9.4 USING and WITH CHECK

Clause	Checks	When it fails
USING	existing rows (read, update, delete)	The row is silently invisible: 0 rows, no error. (Our API turns an update of 0 rows into 404.)
WITH CHECK	new or changed rows (insert, update)	ERROR 42501: new row violates row-level security policy. (Our API turns this into 403.)

9.5 The owner trap and FORCE

Connected as the table owner, the same query returns **every company's rows** without any context, because owners skip RLS. Two defences, use both: never give the app the owner's login, and run `ALTER TABLE ... FORCE ROW LEVEL SECURITY` so even the owner obeys the policies.

9.6 PgBouncer: pooling and passwords

Transaction pooling

PgBouncer lends a client a real PostgreSQL connection **for one transaction only**, then gives it to someone else. That is why the "who is asking" values must be set with `set_config(..., true)` inside the transaction. A plain `SET` would stay on the connection after `COMMIT`, and the next user of that connection could run with another company's context. (Our Go code sets `pgx's QueryExecModeExec`, which uses no cached prepared statements, so it is safe with transaction pooling. PgBouncer's `max_prepared_statements` setting is a second safety net.)

Checking passwords of users that didn't exist a minute ago

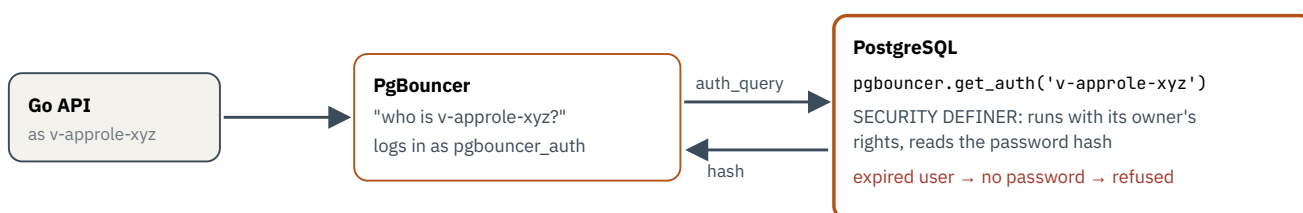


Figure 12. PgBouncer's `auth_query` lets it check temporary users. A fixed user list could never know them.

`pgbouncer_auth` itself cannot read the password table; it can only call that one function, which returns one user's hash and nothing for expired users.

IN OUR LAB

The database is created by `reference/postgres/initdb/` : `01-roles.sql` (roles), `02-schema.sql` (tables, grants), `03-rls.sql` (helpers, policies), `04-pgbouncer-auth.sql` (the lookup function), `05-seed.sql` (2 companies, 5 users, 9 conversations), `06-openbao-admin.sql` (OpenBao's admin role). They run only when the data volume is empty. PgBouncer's settings are in `reference/pgbouncer/pgbouncer.ini` (`pool_mode = transaction`, `auth_query`).

Tested in psql as `app_runtime` : no context → 0 rows; Ana → 3; Sari → 5; insert into the other company → error 42501; update of a colleague's row → `UPDATE 0` ; as owner → all rows, and 0 after `FORCE` ; `EXPLAIN` shows the policies as an extra filter on the query.

FOR PRODUCTION

- Split roles first: `app_owner` for migrations, `app_runtime` for the app. Then enable RLS **one table at a time**, in staging first, with `FORCE` .
- Index `company_id` (and the columns the doors use) on every protected table.
- Views run with their owner's rights and would bypass RLS: create them with `security_invoker = true` .
- A unique constraint can reveal that a row exists in another company (the insert fails). Include `company_id` in such constraints where possible.
- Managed PostgreSQL (TencentDB) has no true superuser; it provides an admin role instead. Check whether that role bypasses RLS, and give the app its own role there too.

OFFICIAL SOURCES

- PostgreSQL: Row Security Policies · `CREATE POLICY` (permissive/restrictive, `USING`/`WITH CHECK`)
- PostgreSQL: `set_config` / `current_setting` · role attributes (`BYPASSRLS`) · `CREATE VIEW` (`security_invoker`)
- PgBouncer: features (pooling modes, what breaks in transaction mode) · PgBouncer: configuration (`auth_query`, `auth_user`)
- TencentDB for PostgreSQL: privileges

10 One request, end to end

IN ONE PARAGRAPH

When a user loads their conversations, the API first checks their login token (a JWT signed with a key from OpenBao). It then opens a transaction, tells PostgreSQL "this is user X from company Y with role Z", and runs a query **without any tenant filter**. PostgreSQL adds the policies, returns only the allowed rows, and forgets the context at `COMMIT`. OpenBao is not called during the request at all.

10.1 Step 0: where "who" comes from

At login, the API signs a **JWT** containing the user's ID, company and role, valid for one hour. The signing key is a random value stored only in OpenBao (`secret/kouventa/app`). Anyone who edits the token breaks the signature, so a user cannot pretend to belong to another company.

10.2 The request path

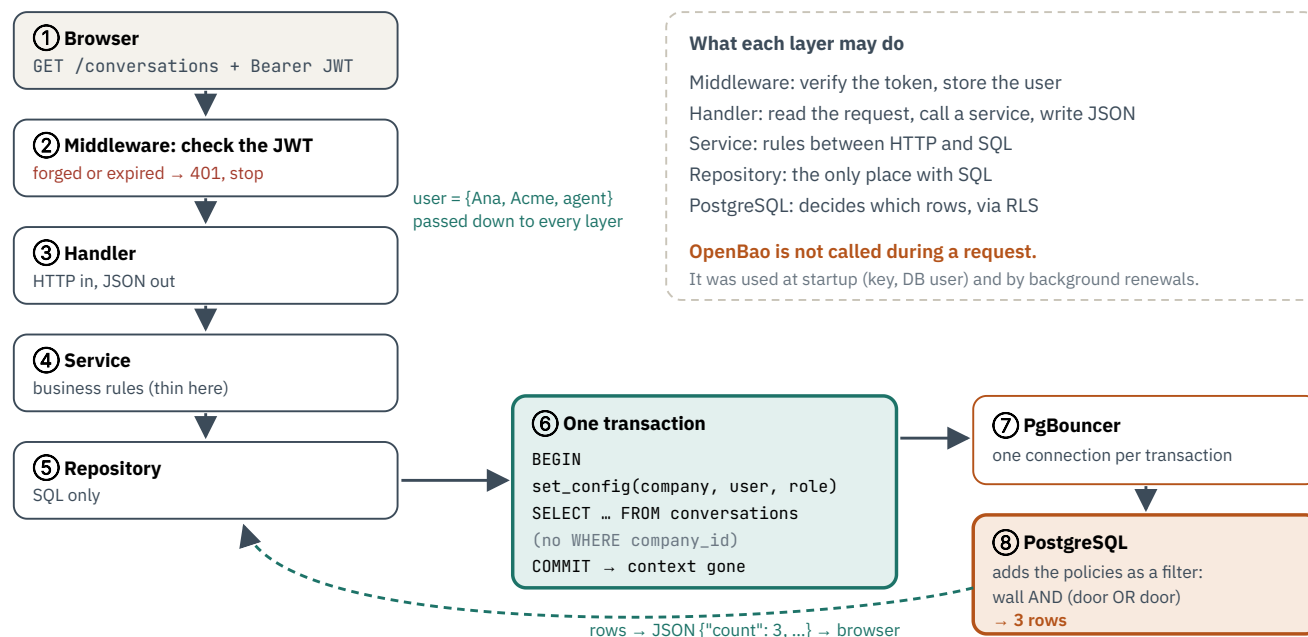


Figure 13. `GET /conversations` from browser to rows. The tenant filter is applied in step ⑧, by the database, not by Go code.

10.3 The design rules behind it

Rule	What goes wrong otherwise
"Who" comes from the verified JWT	Taken from the request body, anyone could send another company's ID.
<code>set_config(..., true)</code> , local to the transaction	A plain <code>SET</code> stays on the pooled connection, and the next request inherits it.
Context and queries in one transaction	Outside a transaction, PgBouncer may run them on different connections, and the query sees no context (0 rows).

Rule	What goes wrong otherwise
No <code>WHERE company_id</code> in application code	One forgotten filter anywhere is a leak; with RLS, the database enforces it everywhere.
The app connects as an <code>app_runtime</code> member	As the owner, RLS is skipped entirely.

10.4 What each kind of attempt produces

Attempt	Stopped by	Result
Edit the JWT to say another company	signature check (step ②)	401, no SQL runs
Update a colleague's conversation	RLS <code>USING</code>	0 rows changed → 404
Create a conversation in another company	RLS <code>WITH CHECK</code>	error 42501 → 403
Code forgets to set the context	helper functions return NULL	0 rows (fail-safe)
App connected as the table owner	nothing, unless <code>FORCE</code>	every company's rows (the trap)

IN OUR LAB

The demo page at `http://localhost:3000` has a button for each attempt above. With PostgreSQL statement logging switched on temporarily, the log shows exactly `BEGIN`, the `set_config` call, the `SELECT` and `COMMIT`, all from the temporary `v-approle-...` user.

Tested: each of the five users saw only their own rows (Dewi 3, Eko 4, Ana and Budi their own plus the unassigned queue, Sari all of Acme); no token → 401; spoofed company → 403; a colleague's row → 404; no context → 0; as owner → rows from both companies.

OFFICIAL SOURCES

- PostgreSQL: Row Security Policies · `set_config` · PgBouncer: SET in transaction pooling
- golang-jwt/jwt v5 · pgx v5 · Fiber

11 Integrating a Go service

IN ONE PARAGRAPH

A service needs about **five calls** to OpenBao: log in, read secrets, get a DB user, renew, and (optionally) ask which node is active. They all happen at startup or in two background jobs, never during a request. With the usual **handler** → **service** → **repository** layers, only one package talks to OpenBao and only one helper sets the RLS context, so existing services change in very few places.

11.1 The calls a service makes

We use the official Go client, github.com/openbao/openbao/api/v2, plus its AppRole helper. The same code works against Vault.

Call	HTTP	Go client	When
Log in	PUT <code>auth/approle/login</code>	<code>client.Auth().Login</code>	startup; after a long pause
Read secrets	GET <code>secret/data/kouventa/app</code>	<code>client.KVv2("secret").Get</code>	startup
Get a DB user	GET <code>database/creds/kouventa-app</code>	<code>client.Logical().Read</code>	startup; each rotation
Renew token / DB lease	PUT <code>auth/token/renew-self</code> , PUT <code>sys/leases/renew</code>	<code>Token().RenewSelf</code> , <code>Sys().Renew</code>	every 30 s (lab)
Active node (status page)	GET <code>sys/leader</code>	<code>client.Sys().Leader</code>	on demand

After login, the client attaches the token to every request automatically. If OpenBao refuses a call because of the policy, the client returns an error with status `403`.

11.2 Layers and packages

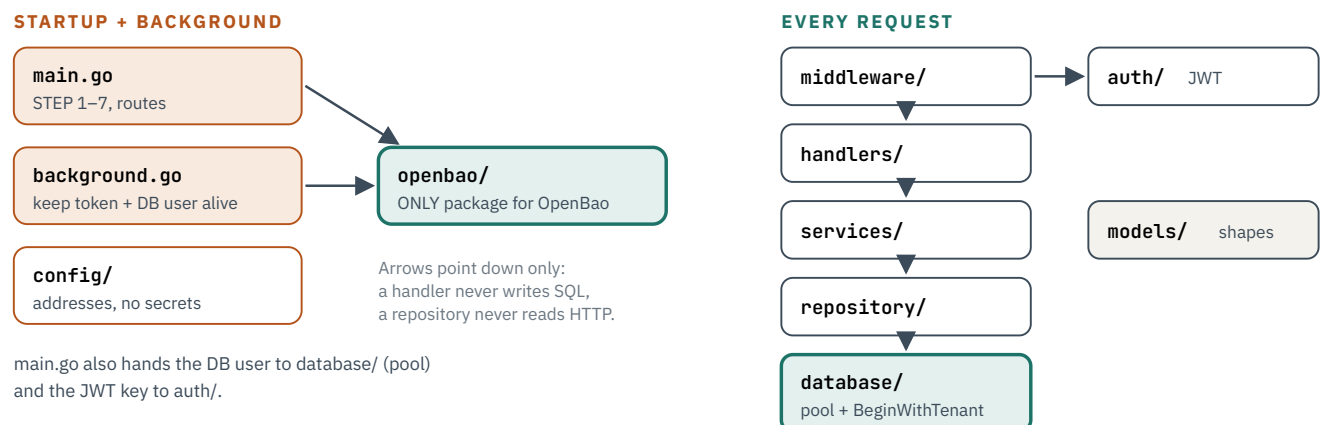


Figure 14. The repository pattern with OpenBao and RLS added. Two packages are new compared with a typical service: **openbao/** and the RLS helper in **database/**.

11.3 Startup, in order

```
api/main.go (outline)

cfg := config.Load() // STEP 1 addresses only, no secrets
openbao.Connect(cfg.OpenBaoAddr) // STEP 2 log in with AppRole
openbao.Login(cfg.RoleIDFile, cfg.SecretIDFile)
secrets, _ := openbao.ReadAppSecrets(cfg.Product) // STEP 3 read secrets
auth.SetSigningKey(secrets.JWTSigningKey)
creds, _ := openbao.GetDBCredentials(cfg.DBRole) // STEP 4 OpenBao creates a DB user now
pool, _ := database.Connect(cfg.DBAddr, cfg.DBName, creds.Username, creds.Password)
database.SetPool(pool, creds.Username) // STEP 5 connect through PgBouncer
go keepTokenAlive(cfg) // STEP 6 background jobs
go keepDatabaseUserAlive(cfg, creds)
app.Listen(":" + cfg.Port) // STEP 7 serve requests
```

Every step stops the program with a clear message if it fails (*fail fast*). Running without secrets would be worse than not starting.

11.4 Keeping things alive

Job	Loop (every 30 s in the lab)
keepTokenAlive	Renew the token. If that fails (e.g. the process was paused longer than the token period), log in again with AppRole.
keepDatabaseUserAlive	Renew the DB lease. If more than 60 s remain, wait. Otherwise (or if renewal failed): get a new DB user → open a new pool → swap it in → the old pool closes 5 s later. If any step fails, keep the old pool and retry next time.

Requests always ask `database.GetPool()` for the current pool, so a swap is invisible to them. A mutex protects the swap, because requests read the pool while the background job replaces it.

11.5 The RLS bridge

```
api/database/tenant.go

func BeginWithTenant(pool *pgxpool.Pool, user models.User) (pgx.Tx, error) {
    tx, err := pool.Begin(ctx)
    ...
    _, err = tx.Exec(ctx, `SELECT set_config('app.company_id', $1, true),
                                set_config('app.user_id', $2, true),
                                set_config('app.role', $3, true)`,
        user.CompanyID, user.ID, user.Role) // from the verified JWT
    ...
    return tx, nil
}
```

```

api/repository/conversation_repository.go

func ListConversations(user models.User) ([]models.Conversation, error) {
    tx, err := database.BeginWithTenant(database.GetPool(), user) // 1. who
    ...
    defer tx.Rollback(ctx) // safety net
    conversations, err := readConversations(tx) // 2. SELECT, no tenant filter
    ...
    err = tx.Commit(ctx) // 3. context disappears
    return conversations, err
}

```

Postgres errors are translated into HTTP only at the top: the handler checks for error code `42501` (`database.IsRLSViolation`) and answers `403`; an update that changed 0 rows becomes `404`.

11.6 Adopting this in an existing service

1. Add an `openbao` package (copy `api/openbao/openbao.go`).
2. At startup: log in, read secrets, and replace each `os.Getenv("X")` for a secret with the value from OpenBao.
3. Database, first step: read a **static role** password from OpenBao (no other code change). Later: dynamic users plus the pool swap.
4. For RLS: in the repository, start every transaction on protected tables with `BeginWithTenant`, using the user from the verified JWT.
5. Run as a periodic-token AppRole; let the deploy pipeline deliver the `secret_id`.

IN OUR LAB

The API in `api/` follows exactly this structure: `main.go`, `background.go`, and the packages `config`, `openbao`, `database`, `models`, `repository`, `services`, `handlers`, `middleware`, `auth`. It is written plainly on purpose, so it reads as a template. `api/cmd/simple/main.go` tells the same story in one file without a web server. `api/README.md` lists every package's job.

Tested (8 Oct 2026): 150 of 150 requests succeeded across 5 automatic DB user rotations; 75 of 75 succeeded while the active OpenBao node was stopped.

OFFICIAL SOURCES

- [OpenBao Go client \(api/v2\)](#) · [AppRole](#) · [leases and renewal](#)
- [pgx pgxpool](#) · [Fiber v3 docs](#)

12 What breaks if... failure scenarios

IN ONE PARAGRAPH

Most failures are less dramatic than they look, because a running service only needs OpenBao for startup and renewals, never for individual requests. The dangerous ones are mistakes that bypass RLS (the owner login, session-level settings) and losing the unsealer's key. This chapter is a quick lookup table for incidents and code reviews.

12.1 Infrastructure

If...	What happens	What to do
The leader node dies	A few seconds of 503 at the load balancer while Raft elects a new leader; then normal. Running apps barely notice.	Nothing urgent. Restart or replace the node; it catches up by itself.
Two of three nodes die	No majority, no leader: OpenBao stops answering. Running apps keep serving requests with their existing DB pool; renewals fail, and trouble starts only when the DB user's lease or the token runs out.	Bring one node back as fast as possible. Longer production TTLs give more time.
The unsealer is down	Nothing, while the nodes keep running. Any node that restarts stays sealed.	Restore the unsealer before restarting nodes.
Everything was off longer than the unseal token's period	Nodes get 403 from the unsealer and stay sealed.	Create the unseal token again, restart the nodes.
The unsealer's key is lost with no backup	The cluster can never unseal again.	Prevent it: back up the unsealer separately and test the restore.
A full restart of the cluster	All nodes auto-unseal; a leader is elected in ~10–30 s (we measured ~15 s). Apps starting in that window get 503.	Apps retry at startup.

12.2 Application and credentials

If...	What happens	What to do
The app is paused longer than its token period (e.g. a VM freeze)	The token expired, so its DB user was revoked too. On waking, renewals fail; the app logs in again and gets a new DB user. It heals itself within a cycle or two.	Nothing; this is why both background jobs have a fallback.
The app's <code>secret_id</code> expires while it runs	No effect: the periodic token keeps working. The next restart fails to log in.	The deploy pipeline delivers a fresh <code>secret_id</code> with every deploy.
The app asks for another product's secret	403 from OpenBao.	Expected: the policy works.

If...	What happens	What to do
A full reset of a test cluster, but old saved tokens kept	Setup scripts trust stale files and fail with confusing errors ("Vault is sealed", login 400).	Delete the saved tokens and login files, then init and configure again.

12.3 Row Level Security (code review checklist)

If a developer...	Result	Severity
forgets <code>BeginWithTenant</code> in a new query	0 rows (fail-safe)	visible bug, no leak
uses plain <code>SET app.company_id = ...</code>	context can leak to the next request on a pooled connection	cross-tenant leak
connects the app as the table owner	RLS skipped: every company's rows	cross-tenant leak (unless <code>FORCE</code>)
takes <code>company_id</code> from the request body	anyone can act as another company	cross-tenant leak
adds a view on a protected table	the view runs with its owner's rights and may skip RLS	leak, unless <code>security_invoker</code>
adds a new table with tenant data	no policy, no RLS	leak until RLS is enabled on it

IN OUR LAB

Every row in 12.1 and the first two rows of 12.2 were reproduced on purpose in the lab, apart from losing the unsealer's key. The stale-files case happened by accident during testing.

13 Production deployment

IN ONE PARAGRAPH

Production is the lab design with real VMs, TLS, firewalls, people-held keys, longer TTLs and scheduled backups. Start with 3 OpenBao nodes plus one small unsealer VM and a load balancer, all in a private network across availability zones; grow to 5 nodes. The real cost is operations time: certificates, upgrades, backups and key holders.

13.1 Target architecture

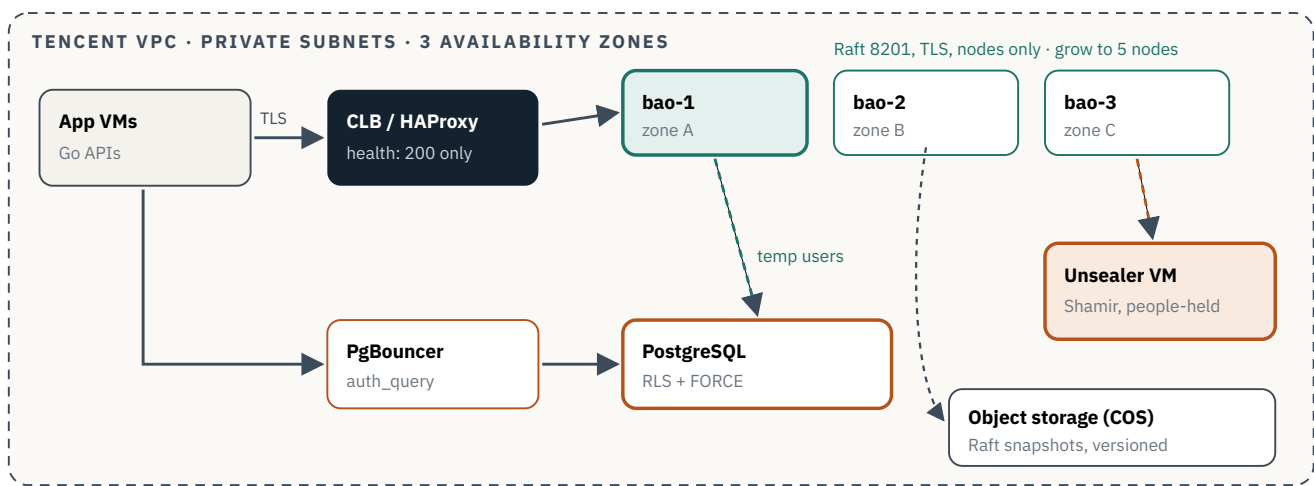


Figure 15. Recommended production layout on Tencent.

13.2 Decisions

Decision	Recommendation	Why
Storage backend	Raft (integrated)	Tested end to end, no external dependency. OpenBao 2.7 also offers a PostgreSQL backend and recommends it for newcomers; revisit if we run a managed HA PostgreSQL that is separate from the product databases (OpenBao must not depend on the database it creates users for).
Unsealing	Transit + unsealer unsealed by Shamir	Chapter 5. Tencent KMS has no seal; HSM via plugin is untested.
Node count	3 to start, 5 when products depend on it	Survive 1 or 2 failures (chapter 6).
Size per node	Start small: 2 vCPU / 8 GB, SSD ≥ 100 GiB	HashiCorp's "small" profile is 2–4 cores, 8–16 GB, 3,000+ IOPS; avoid burstable instance types. A 100 GiB Tencent SSD gives ≈ 4,800 IOPS.
One cluster or one per cloud	One central cluster to start, a path or namespace per product	Apps only call OpenBao at startup and on renewals, so cross-cloud latency matters little; the link must be reliable and always TLS.

13.3 Lab vs production

Area	Lab	Production
Machines	containers on one laptop	VMs across zones
Transport	HTTP	TLS everywhere (API, cluster, unsealer, PostgreSQL)
Unsealer	static key from compose	own VM, Shamir keys with ≥ 3 people, separate backups
Unseal token	fixed name, 24h period, in compose	random, longer period, root-only file, alert on time left
Root tokens	JSON files	revoked after setup; recovery keys PGP-encrypted to several people
App login	<code>secret_id</code> file, 24h, reusable	delivered by the pipeline, wrapped, single-use, bound to app IPs
TTLs	token 2m; DB user 2m / max 6m	e.g. token 1h; DB user 1h / max 24h
RLS	no <code>FORCE</code> (to show the trap)	<code>FORCE</code> on; the app never uses the owner login
Backups	manual snapshot script	scheduled snapshots to object storage; restore tested regularly
People	root token	OIDC/userpass logins with least-privilege admin policies

13.4 Running it

- **Install:** official packages, checksums verified. Disable or encrypt swap, disable core dumps, dedicated VMs, not running as root.
- **Monitoring:** Prometheus metrics at `/v1/sys/metrics?format=prometheus`. Alert on a sealed or down node, leader changes, Raft peer count, certificate expiry, lease counts, audit-log failures, and the unseal token's time left.
- **Audit:** configure the audit device in each node's config; ship the log off the VM. Secret values are hashed in it.
- **Backups:** snapshots on a timer, uploaded encrypted and versioned. Back up the unsealer separately. Test a restore on a spare VM.
- **Upgrades:** read every release note in between, snapshot first, then standbys one at a time, `step-down`, and the old leader last. Minor releases come every few months, with patch releases in between.
- **OpenBao's DB admin role:** `GRANT app_runtime TO openbao_admin WITH ADMIN OPTION, INHERIT FALSE, SET FALSE;` so the role that creates users cannot use their rights (PostgreSQL 16+).
- **Configuration as code:** node configs, policies and setup scripts live in git and are reviewed like code. The UI is for exploring only.

SECURITY CHECKLIST

TLS everywhere · swap off or encrypted · 8201 open only between nodes · root token revoked · recovery and unsealer keys split across people · people log in with OIDC/userpass · one AppRole and policy per service, `secret_id` limited (TTL, uses, CIDR) · audit log shipped off the VM · snapshots with tested restore · monitoring and alerts · all configuration in git.

OFFICIAL SOURCES

- [OpenBao: install and hardening](#) · [storage backends](#) · [PostgreSQL storage](#) · [audit configuration](#) · [telemetry](#)
- [OpenBao: upgrading HA clusters](#) · [support vendors](#)
- [HashiCorp Raft reference architecture](#) · [Vault production hardening](#) (applies equally)
- [Tencent CVM cloud disks \(IOPS\)](#) · [Tencent CLB health checks](#) · [Tencent COS](#)

14 Rollout plan, risks and open questions

IN ONE PARAGRAPH

Roll out in small, reversible steps: first a staging cluster, then one service's `.env`, then all services, then database passwords, and only then RLS and dynamic users. Two tracks can start immediately because they don't depend on OpenBao: splitting the database roles, and reviewing Firebase rules.

14.1 Phases

#	Work	Depends on
1	Staging OpenBao cluster on Tencent from our reference files, with TLS. The team practises init, failover and restore.	—
2	First service in staging: <code>.env</code> → OpenBao, AppRole login at startup.	1
3	Production cluster; move each service's <code>.env</code> , then delete the file. Rotate any value ever committed to git.	2
4	Database static role : OpenBao rotates the shared login's password.	3
5	Split database roles: <code>app_owner</code> (migrations) and <code>app_runtime</code> (app).	— (can start now)
6	RLS table by table, staging first, with <code>FORCE</code> ; RLS tests in CI.	5
7	Dynamic DB users (<code>IN ROLE app_runtime</code>) and PgBouncer <code>auth_query</code> .	4, 5
8	Firebase: custom claims, rules tests in CI, review backend endpoints that use the Admin SDK.	— (can start now)

14.2 Risks

Risk	Mitigation
OpenBao down → services cannot start (running ones continue until leases end)	HA cluster, longer production TTLs, alerts
Unsealer or its key lost → cluster cannot unseal	Separate backups; Shamir keys with ≥ 3 people; tested restore
Recovery keys and root token lost	Split keys; documented break-glass procedure
Community-only support	Commercial vendors exist; Vault-compatible API keeps a fallback
Version churn (2.7 moved several features to plugins)	Read release notes; staging first; snapshot before upgrades
RLS rollout returns empty results where context is missing	One table at a time in staging; RLS tests in CI

Risk	Mitigation
Firebase Admin SDK bypasses rules	Authorization in Go; code review checklist

14.3 To verify before production

1. Tencent Cloud HSM with OpenBao's PKCS#11 plugin (would remove manual unsealing of the unsealer).
2. TencentDB for PostgreSQL, if used: whether its admin account can do `CREATEROLE` -based dynamic users and the PgBouncer lookup function.
3. Network path from AWS/Azure VMs to a Tencent cluster (VPN or private link).
4. Who holds the recovery keys and the unsealer's Shamir keys (at least 3 people).
5. Run the Firebase rules tests once (written against current docs, not yet executed; chapter 15).

15 Firebase: Security Rules

IN ONE PARAGRAPH

For reads and writes coming from our web and Flutter apps, **Firestore Security Rules** do the same job as PostgreSQL RLS: every request is checked against rules that know who the user is. Two big differences: the Go backend's **Admin SDK bypasses all rules**, and rules **are not filters**, so a query that *could* return a forbidden document fails completely instead of being trimmed.

15.1 Who is checked where

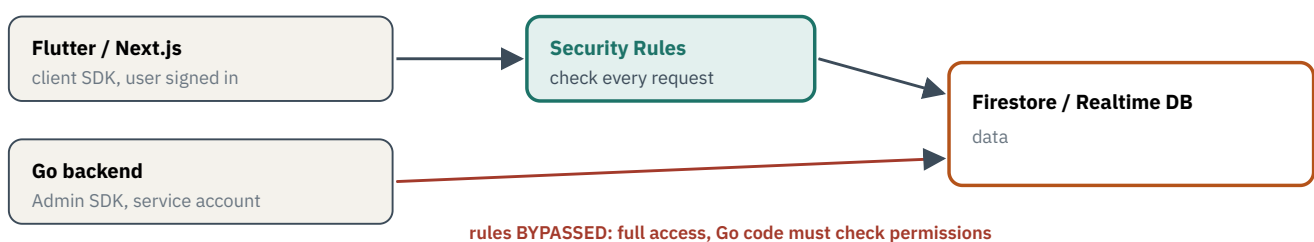


Figure 16. Rules protect client access only. Backend endpoints that read Firebase for a user must authorize that user in Go, like any API.

15.2 PostgreSQL RLS and Firebase Rules side by side

Concept	PostgreSQL RLS	Firestore Rules	Realtime DB Rules
Rules live in	the database (<code>CREATE POLICY</code>)	<code>firestore.rules</code> , deployed with the Firebase CLI	<code>database.rules.json</code>
"Who is asking"	<code>set_config</code> per transaction	<code>request.auth.uid</code> , <code>request.auth.token.<claim></code>	<code>auth.uid</code> , <code>auth.token.<claim></code>
Default	deny		
Existing data (≈ USING)	<code>USING (...)</code>	<code>resource.data</code>	<code>data</code>
New data (≈ WITH CHECK)	<code>WITH CHECK (...)</code>	<code>request.resource.data</code>	<code>newData</code>
Combining rules	permissive OR, restrictive AND	several <code>allow</code> → OR	cascade: access granted higher up can't be removed lower down
Query without the filter	silently filtered	whole query fails	whole read fails
Field-level rules	column grants or views	<code>diff().affectedKeys().hasOnly([...])</code>	rules per child key
Bypass	superuser, owner, <code>BYPASSRLS</code>	Admin SDK	Admin SDK

Concept	PostgreSQL RLS	Firestore Rules	Realtime DB Rules
Testing	SQL as the runtime role	Emulator + <code>@firebase/rules-unit-testing</code>	

15.3 Tenant and role: custom claims

Firebase Auth tokens can carry **custom claims**, e.g. `{ company_id: "acme", role: "agent" }`. Set them **only from the backend** (Admin SDK, `SetCustomUserClaims` in Go). Keep them small (limit 1,000 bytes): IDs and roles, not data. Clients receive new claims with their next ID token (after sign-in, a forced refresh, or the hourly refresh), so force a refresh after changing someone's role. This is Firebase's equivalent of our signed JWT and `set_config`.

15.4 The same support-desk rules in Firestore

```
reference/firebase/firestore.rules (excerpt)
match /companies/{companyId}/conversations/{conversationId} {
  // wall: own company · doors: supervisor, own, or unassigned
  allow read: if inCompany(companyId)
    && (isSupervisor()
      || isMine(resource.data)
      || resource.data.assigned_to == null);

  // agents may change only the status of their own conversations
  allow update: if inCompany(companyId)
    && (isSupervisor() || isMine(resource.data))
    && validStatus(request.resource.data.status)
    && (changedOnly(['status'])
      || (isSupervisor() && changedOnly(['status', 'assigned_to'])));

  allow delete: if false;
}
```

Because rules are not filters, an agent's app must query with the same condition the rule checks, e.g. `where('assigned_to', '=', uid)`. Without it, the whole query is denied.

15.5 Gotchas

- **Realtime DB cascade:** a `.read: true` high in the tree opens everything below it. Grant at the lowest level possible.
- **Missing fields** in Firestore rules cause errors, and errors deny. Store explicit `null` (we do for `assigned_to`).
- **Lookups are limited:** `get()` / `exists()` calls to other documents are capped (10 per single-document request or query, 20 for batches and transactions). Prefer claims for frequent checks.
- **The Admin SDK's service-account key is a secret.** Store it in OpenBao, not in `.env`.
- **App Check** blocks requests that don't come from our real apps (abuse protection). It complements rules; it does not replace authorization.
- Firebase **Data Connect** is now called **SQL Connect**: managed PostgreSQL behind a Firebase API, with authorization per query (`@auth`). Its docs do not describe it as PostgreSQL RLS; only relevant if clients

should reach PostgreSQL directly.

IN OUR LAB

`reference/firebase/` contains `firestore.rules`, `database.rules.json` and emulator tests (`test/rules.test.mjs`, run with `npm test`; needs Node 22+ and Java 21+). They encode the same cases as the PostgreSQL demo. One Realtime DB limitation: agents can list their own conversations, but not the unassigned queue (a rule allowing `equalTo(null)` would also allow unfiltered reads); use a sentinel value or a separate queue node. **The tests have not been executed yet:** the emulator download was blocked in the build environment. Run them once before relying on them.

RECOMMENDATION

1. Keep Security Rules on every collection or path the apps read: tenant claim + role claim, deny by default, tests in CI with the emulator.
2. Set `company_id` and `role` custom claims from the Go backend at login or role change.
3. Treat the Admin SDK like a superuser connection: check permissions in Go.
4. Keep the service-account key in OpenBao.

OFFICIAL SOURCES

- [Firestore: rules and queries \(rules are not filters\)](#) · [Firestore: writing conditions](#) · [Firestore rules: get started \(server libraries bypass rules\)](#)
- [Realtime DB: rules syntax \(cascade\)](#) · [Realtime DB: conditions](#) · [Realtime DB: Admin SDK](#)
- [Custom claims](#) · [Rules unit tests](#) · [SQL Connect \(formerly Data Connect\)](#)

A The lab: repository map and test results

Repository map (`secrets-rls-lab`)

Path	What it is
<code>reference/docker-compose.yml</code>	The production-like stack: unsealer, 3 OpenBao nodes, HAProxy, PostgreSQL, PgBouncer, pgAdmin, admin tools
<code>reference/unsealer/</code>	<code>unsealer.hcl</code> (static seal, lab only) and <code>setup.sh</code> (transit key, policy, token)
<code>reference/openbao/</code>	<code>bao-1/2/3.hcl</code> node configs; <code>policies/kouventa-app.hcl</code>
<code>reference/haproxy/haproxy.cfg</code>	Load balancer: health check, active node only
<code>reference/scripts/</code>	<code>init.sh</code> , <code>configure.sh</code> , <code>import-env.sh</code> , <code>snapshot.sh</code> , <code>app-check.sh</code>
<code>reference/postgres/initdb/</code>	Roles, schema, RLS policies, PgBouncer lookup, seed data, OpenBao's admin role
<code>reference/pgbouncer/</code>	<code>pgbouncer.ini</code> (transaction pooling, <code>auth_query</code>)
<code>reference/firebase/</code>	Firestore and Realtime DB rules + emulator tests
<code>api/</code>	Go + Fiber API (handler → service → repository), demo page; <code>api/cmd/simple/</code> = one-file version
<code>learning-lab/</code>	Earlier learning lab in dev mode: OpenBao and Vault side by side (the comparison tests)
<code>reference/README.md</code>	Step-by-step guide to run and test everything

Test results

Milestone	What we proved
M1	AppRole login; secrets from KV instead of <code>.env</code> ; policy allows own path, denies other products and the admin path; namespaces work in OpenBao
M2	DB credentials from OpenBao: static role (rotates an existing login) and dynamic users
M3	RLS through PgBouncer per company, user and role; spoofed insert blocked; no context → 0 rows; owner trap and the <code>FORCE</code> fix
M4	Same scripts and Go code on Vault 2.1.1; image and memory measurements; namespaces are Enterprise-only in Vault
M5	3-node cluster + HAProxy + transit auto-unseal; <code>.env</code> import; dynamic DB users with renewal and rotation; leader killed with no failed requests; full restart → auto-unseal with data intact; snapshot restore; audit log
Re-test (8 Oct)	Rewritten Go API: 150/150 requests across 5 DB user rotations; 75/75 with the active node stopped; app-check: own secret ✓, admin × 403, other product × 403, write × 403, temporary user ✓

Running it (short version)

```
PowerShell, from the repository root

# start the stack
docker compose -f reference/docker-compose.yml up -d --build
# once per new cluster: create the keys, then the configuration (safe to re-run)
docker compose -f reference/docker-compose.yml run --rm tools /scripts/init.sh
docker compose -f reference/docker-compose.yml run --rm tools /scripts/configure.sh
# run the API → http://localhost:3000
cd api; go run .
```

If the stack was stopped for more than 24 hours, the nodes' unseal token has expired: run `up -d` again (it recreates the token) and restart the three nodes. After `down -v`, delete `reference/.secrets/*.json` before init.

B Command cheat sheet

Command	Does
<code>bao status</code>	Seal type, sealed or not, HA mode (active/standby), Raft index
<code>bao operator init</code>	Initialize a new cluster (once)
<code>bao operator seal / unseal</code>	Seal a node by hand / give an unseal (or recovery) key share
<code>bao operator step-down</code>	Make the leader hand over (before maintenance)
<code>bao operator raft list-peers</code>	Cluster members, leader, voters
<code>bao operator raft autopilot state</code>	Health of each member
<code>bao operator raft remove-peer <id></code>	Remove a dead node permanently
<code>bao operator raft snapshot save restore <file></code>	Back up / restore the whole cluster
<code>bao secrets enable -path=secret kv-v2</code>	Mount an engine
<code>bao kv put get secret/<product>/app</code>	Write / read static secrets
<code>bao kv metadata get ...</code>	Versions of a secret
<code>bao policy write <name> <file></code>	Store a policy
<code>bao write auth/approle/role/<role> ...</code>	Configure an app login
<code>bao read database/creds/<role></code>	Create a temporary DB user
<code>bao lease renew revoke <lease_id></code>	Extend / end a lease (revoke drops the DB user)
<code>bao token lookup [token]</code>	Policies, period, time left, last renewal
<code>bao path-help <path></code>	List an engine's routes
<code>any command -output-curl-string</code>	Show the HTTP request instead of sending it
<code>any command -output-policy</code>	Show the policy that command would need

Environment variables read by the CLI: `BAO_ADDR` (where to send requests) and `BAO_TOKEN` (which token to attach). The Vault CLI uses `VAULT_ADDR` and `VAULT_TOKEN`. Full reference: openbao.org/docs/commands.